M2S-01: Step 1 — Discover: Understand SaaS Differences

> **Series:** M2S — Managed to SaaS Migration | **Notebook:** 1 of 9 | **Phase:** Plan | **Step:** Discover | **Created:** March 2026 | **Last Updated:** 07/30/2026

The first step in any Managed-to-SaaS migration is understanding what you are moving to and why. This notebook helps you document the benefits of Dynatrace SaaS for your organization, take inventory of your current Managed environment, and confirm your use cases and goals for the upgrade.

> **M2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan: 1. Discover** | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run:** 7. Enable | 8. Expand | 9. Optimize

Sprint 1.337 (April 2026) Updates Affecting M2S

Three sprint-1.337 changes affect a Managed → SaaS migration:

1. **OneAgent primary fields/tags at source** — Latest Dynatrace SaaS tenants gain primary fields (`dt.security_context` , `dt.cost.costcenter` , `dt.cost.product`) and customer-defined primary tags as top-level attributes. Consider this in the M2S-03 (Design) step when deciding what tag/security-context model to recreate post-migration.
2. **Configuration API → Settings v2 acceleration** — Dynatrace Managed exposes the older Configuration API for many resources; the SaaS target's Settings v2 surface now covers more of those endpoints. Migration tooling (M2S-04/05) should target Settings v2 wherever possible. Plan for the legacy/v2 split where Managed has a unique Configuration API endpoint.
3. **Platform tokens** for new automation. Classic `dt0c01` still works for legacy paths, but new SaaS pipelines should default to `dt0s16` Platform tokens (`Authorization:`

`Bearer …`). Note: `dt0s01` is a separate SCIM/account-management token — it is NOT a Platform Token prefix and should not be used for environment-level automation.

Extensions review (ToDo #1) — if Managed carries custom **Extensions Framework 1.0** extensions, migration to SaaS is the natural moment to rebuild them as **Extensions 2.0**, the current extensions framework. EF1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31); JMX and PMI EF1.0 are deprecated but supported past that date on request. Extensions 2.0 are managed via the Dynatrace API Application → Extensions surface.

Table of Contents

Inventory Categories

A complete discovery must cover ALL of these categories — missing any one will create surprises during execution:

Category	What to Inventory	Migration Method
Configurations and settings	All environment-level settings, entity-level settings	Automated via SaaS Upgrade Assistant
Credential Vault	All stored credentials, certificates	Manual — secrets cannot be exported

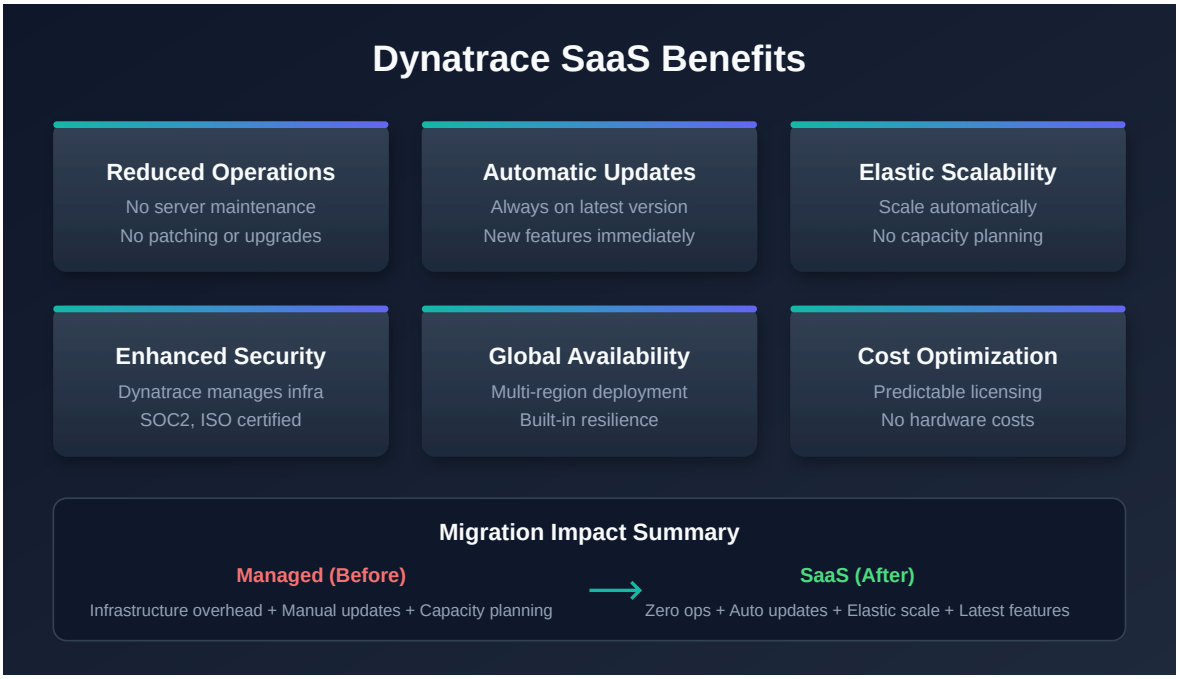
Category	What to Inventory	Migration Method
API tokens	All tokens and their scopes	Manual — recreate with minimal scopes
Extensions	OneAgent extensions, ActiveGate extensions — are they current or need upgrade?	Manual — evaluate for Extensions 2.0
External and custom sources	Cloud integrations (AWS/Azure/GCP), Kubernetes integration, log ingest, custom metrics	Manual — reconfigure each source
Other integrations	ITSM, CMDB, reports, data lakes, CI/CD pipelines	Manual — update endpoints and tokens
Monitoring components	OneAgent instances (hosts, PaaS, K8s/OpenShift), ActiveGate instances (routing, extensions, synthetic, zRemote)	Automated (agent redirect)
Dashboards	All dashboards, their owners, and management zone filters	Automated via SaaS Upgrade Assistant

> **Tip:** Consider environment clean-up during discovery. Excessive or legacy configuration items can be left behind. Most items like tags or management zones migrate in full, but others like dashboards should be reviewed and migrated selectively.

Prerequisites

Requirement	Details
Dynatrace Managed	Active Managed environment with administrator access
Dynatrace SaaS Tenant	Provisioned SaaS environment (or planning to provision)
API Tokens	Tokens with `entities.read`, `settings.read` scopes on Managed

Requirement	Details
Stakeholder Access	Ability to document and share findings with decision-makers



1. Why Upgrade to SaaS

Moving from Dynatrace Managed to SaaS is not just a hosting change — it unlocks a fundamentally different platform architecture. Understanding these differences helps you build the business case and set expectations with stakeholders.

Infrastructure and Operations

Area	Managed	SaaS
Cluster management	Customer-managed servers, storage, networking	Fully managed by Dynatrace
Scaling	Manual capacity planning and provisioning	Automatic scaling with licensing
Updates	Manual cluster patching (scheduled downtime)	Automatic bi-weekly updates (zero downtime)
Availability	Customer-managed HA/DR	99.5%+ SLA with built-in redundancy

Area	Managed	SaaS
Security patches	Customer responsibility to apply	Automatic, managed by Dynatrace

Platform Capabilities

SaaS provides capabilities that are not available on Managed:

Capability	Description
Grail	Petabyte-scale data lakehouse — unified querying across logs, metrics, traces, events, and entities
DQL	Dynatrace Query Language — context-aware queries replacing USQL
Notebooks	Interactive, collaborative analysis documents (you are reading one now)
AppEngine	Build and deploy custom Dynatrace apps
AutomationEngine	Advanced workflow engine for alerting, remediation, and orchestration
Dynatrace Assist	AI-powered assistant for natural-language queries and analysis
OpenPipeline	Custom data processing, routing, and enrichment at ingest
Business Analytics	Full business event tracking and conversion funnel analysis

Operational Cost Reduction

Migrating to SaaS eliminates operational overhead associated with running Managed clusters:

- No hardware procurement, racking, or lifecycle management
- No cluster OS patching or Dynatrace server upgrades
- No capacity planning for Cassandra, Elasticsearch, or transaction storage
- No backup/restore procedures for cluster data
- No need for dedicated Managed cluster administrators

Enhanced Security and Compliance

Compliance	Coverage
SOC 2 Type II	Annual audit of security controls
ISO 27001	Information security management certification
HIPAA	Health data handling compliance
FedRAMP	US government security authorization (select regions)
Automatic patching	Security vulnerabilities addressed without customer action

2. Review Your Current Environment

Before planning the migration, take a complete inventory of what exists in your Managed environment. This inventory serves two purposes: sizing your SaaS license and identifying configurations that need migration.

> **Note:** DQL queries below run against your **SaaS tenant** (where entity data has been synced or where agents are already reporting). For Managed-only environments, use the **Entities API v2** or **Dynatrace UI** alternatives shown after each query.

Host Inventory

```
// Count total monitored hosts
fetch dt.entity.host
| summarize hostCount = count()

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST" | summarize hostCount = count()
// Caveat: Smartscape counts CURRENT live topology and can report fewer hosts
than the
// classic entity store, which retains monitored hosts not in active topology.
For a
// pre-migration discovery inventory, keep the classic entity-store count
above.
```

```
// Hosts grouped by OS type
fetch dt.entity.host
| summarize hostCount = count(), by:{osType}
| sort hostCount desc

// Smartscape equivalent (deprecated dt.entity.* still functional):
//   smartscapeNodes "HOST" | summarize hostCount = count(), by:{os.type} |
sort hostCount desc
// Field map: osType -&gt; os.type (value form also changes, e.g. LINUX -&gt;
OS_TYPE_LINUX).
// The live-topology count caveat from the previous cell applies here too.
```

Managed alternative (Entities API v2):

```
# Total host count
curl -s "https://{managed-url}/api/v2/entities?
entitySelector=type(HOST)&amp;pageSize=1" \
  -H "Authorization: Api-Token {TOKEN}" | jq '.totalCount'
```

Service Inventory

```
// Count total monitored services
fetch dt.entity.service
| summarize serviceCount = count()

// Smartscape equivalent (deprecated dt.entity.* still functional):
//   smartscapeNodes "SERVICE" | summarize serviceCount = count()
// Caveat: Smartscape reflects CURRENT live topology and typically reports
fewer services
// than the classic entity store (short-lived / inactive services are
omitted). For a
// pre-migration discovery inventory, keep the classic entity-store count
above.
```

```
// Services grouped by technology
fetch dt.entity.service
| summarize serviceCount = count(), by:{serviceType}
| sort serviceCount desc

// Smartscape equivalent (deprecated dt.entity.* still functional):
//   smartscapeNodes "SERVICE" | summarize serviceCount = count(), by:
{dt.service.sdv1_type} | sort serviceCount desc
// Field map: serviceType -&gt; dt.service.sdv1_type (same value labels, e.g.
WEB_REQUEST_SERVICE).
// The live-topology count caveat applies here too.
```

Application Inventory

```
// Count monitored web applications
smartscapeNodes "FRONTEND"
| filter frontend.type == "web"
| summarize appCount = count()

// Smartscape (preferred, verified 07/2026): dt.entity.application maps to the
FRONTEND node,
// filtered on frontend.type == "web" (mobile apps are the same node with
frontend.type ==
// "mobile"). This corrects an earlier note here that claimed no Smartscape
equivalent existed.
// FRONTEND also carries id_classic holding the APPLICATION-* id, for joining
migrated and
// unmigrated queries. Unlike ActiveGate, `fetch dt.entity.application` does
still work and remains
// a genuine fallback — but it reads the classic entity store, which can
retain entities Smartscape
// (live topology) no longer lists, so cross-check if the two counts disagree.
// Classic fallback: fetch dt.entity.application | summarize appCount =
count()
```

Synthetic Test Inventory

```
// Count synthetic monitors (browser monitors; add HTTP_MONITOR for a full
synthetic inventory)
smartscapeNodes "BROWSER_MONITOR"
| summarize syntheticCount = count()

// Smartscape (preferred, verified 07/2026): dt.entity.synthetic_test maps to
the BROWSER_MONITOR
// node (individual steps are a separate BROWSER_MONITOR_STEP node). HTTP
monitors are HTTP_MONITOR,
// multi-protocol monitors NETWORK_AVAILABILITY_MONITOR, private locations
SYNTHETIC_LOCATION. This
// corrects an earlier note here that claimed no Smartscape equivalent
existed. Unlike ActiveGate,
// `fetch dt.entity.synthetic_test` does still work and remains a genuine
fallback — it reads the
// classic entity store, which can retain entities Smartscape (live topology)
no longer lists.
// Classic fallback: fetch dt.entity.synthetic_test | summarize syntheticCount
= count()
```

```
// Synthetic monitors by type (HTTP, browser, multi-protocol)
// dt.entity.synthetic_test does not expose a 'type' field in Grail entity
store;
// use dt.synthetic.events to count distinct monitors by execution event type.
fetch dt.synthetic.events, from:-1h
| filter in(event.type, {"http_monitor_execution",
"browser_monitor_execution", "multiprotocol_monitor_execution"})
| summarize testCount = countDistinct(dt.synthetic.monitor.id), by:
{event.type}
| sort testCount desc

// Note: event.type values map to monitor types:
//   http_monitor_execution      → HTTP monitor
//   browser_monitor_execution   → Browser (clickpath) monitor
//   multiprotocol_monitor_execution → Network availability (TCP/ICMP/DNS)
monitor
```

OneAgent Version Assessment

Understanding your OneAgent version spread is important because SaaS supports a rolling 9-month (Standard) / 12-month (Enterprise) version window. Agents older than this window must be updated before or during migration.

```
// OneAgent versions across hosts
fetch dt.entity.host
| summarize hostCount = count(), by:{installerVersion}
| sort hostCount desc

// No Smartscape equivalent: installerVersion (OneAgent version) is not a
Smartscape node
// field, so this agent-version distribution stays on the classic entity
store.
```

ActiveGate Assessment

```
// ActiveGate inventory
smartscapeNodes "ACTIVEGATE"
| summarize activeGateCount = count()

// Correction (verified 07/2026): this cell previously ran `fetch
dt.entity.active_gate` and
// carried a note claiming Smartscape had no ActiveGate node. Both were wrong.
There is NO classic
// ActiveGate entity type in any spelling (active_gate,
environment_active_gate,
// environment_activegate), so the classic query returned zero rows in every
tenant —
// indistinguishable from "no ActiveGates deployed". `smartscapeNodes
"ACTIVEGATE"` (no
// underscore) is the working path, and it works on tenants today.
// Field maps: entity.name → name, softwareVersion → dt.active_gate.version,
// networkZone → dt.network_zone.id.
// ActiveGate 1.343 (published 07/15/2026, staged tenant rollout from
07/28/2026) deprecates
// GET /api/v2/activeGates, /api/v2/activeGates/{agId} and
/api/v2/activeGates/groups in favour of
// this same Smartscape node — the classic entity and the classic REST
endpoints were retired as one
// move. If you need a REST surface in the meantime, the classic Entities API
v2 selector
// (GET /api/v2/entities?entitySelector=type("ENVIRONMENT_ACTIVE_GATE")) is a
different surface and
// may still respond; the DQL `fetch dt.entity.*` form does not.
```

Environment Size Summary

Record your findings in this table:

Entity Type	Count	Notes
Hosts	___	Include OS breakdown
Services	___	Include technology breakdown
Applications	___	Web and mobile
Synthetic Tests	___	HTTP, browser, scripted
ActiveGates	___	Environment and cluster
Management Zones	___	Will migrate as-is
Dashboards	___	Classic dashboards migrate; rebuild as modern recommended

> **Important:** Dynatrace recommends keeping a single environment at roughly **25,000 hosts of typical load**; the platform can technically scale higher, but beyond that threshold plan to split host-unit quota across multiple SaaS tenants/environments. This is a planning guideline, not a hard cap — discuss tenant topology with your Dynatrace account team before proceeding.

3. Confirm Use Cases and Goals

Every migration needs clearly documented motivations and success criteria. Use the table below to identify which benefits matter most to your organization and align them with concrete goals.

Common Migration Motivations

Motivation	SaaS Benefit	Success Metric
Eliminate cluster maintenance	Fully managed infrastructure	Zero hours spent on cluster patching
Access new capabilities	Grail, Notebooks, AppEngine, Dynatrace Assist	Teams actively using SaaS-exclusive features

Motivation	SaaS Benefit	Success Metric
Improve availability	99.5%+ SLA	Reduced monitoring downtime incidents
Reduce operational cost	No hardware, patching, capacity planning	Measured reduction in Managed infrastructure spend
Enhance security posture	Managed infrastructure, automatic security patches	Faster time-to-patch for vulnerabilities
Simplify compliance	SOC 2 Type II, ISO 27001, HIPAA	Inherited compliance controls
Consolidate tenants	Single SaaS platform	Fewer environments to manage
Enable self-service	IAM policies, Grail permissions	Teams can query their own data without admin requests

Documenting Your Goals

For each selected motivation, document:

1. **Current pain point** — What problem does this solve today?
2. **Expected outcome** — What does success look like after migration?
3. **Measurement** — How will you verify the goal was achieved?
4. **Timeline** — When should this benefit be realized (during migration or post-migration)?

> **Tip:** Share this goals document with stakeholders before proceeding to Step 2 (Strategize). Alignment on goals prevents scope creep and sets realistic expectations.

4. SaaS Upgrade Assistant Overview

The **SaaS Upgrade Assistant** is Dynatrace's primary migration tool for Managed-to-SaaS upgrades. Understanding its capabilities early helps you plan what will be automated versus what requires manual effort.

How It Works

1. **Export** — Download configuration archive from your Managed Cluster Management Console
2. **Upload** — Import the archive into the SaaS Upgrade Assistant app on your target SaaS tenant
3. **Review** — Configurations are grouped by type with error highlighting for incompatible items
4. **Edit** — Fix failed or incompatible configurations individually or in bulk
5. **Deploy** — Push selected configurations to your SaaS environment
6. **Track** — Monitor progress and download deployment result reports (CSV)

Key Features

Feature	Description
Automated configuration import	Most settings and dashboards migrate automatically
Selective import	Smart dependency-aware import — choose what to deploy per wave
Dashboard ownership migration	Updates dashboard owners from Managed to SaaS user identifiers
Entity ID adjustment	Handles entity ID changes between environments
Bulk editing	Edit and correct failed configurations in batch
Preview changes	Review all changes before deploying
Progress tracking	Real-time status with CSV export of results

Requirements

Requirement	Details
Managed version	No fixed minimum in the SaaS Upgrade Assistant docs — align to the same *major* version as the target SaaS tenant (verify the current requirement in docs)

Requirement	Details
Version alignment	Export from the same **major** version as the target SaaS environment to avoid false-positive migration failures (docs example: Managed 1.284.x → SaaS 1.284.x)
IAM policy	`upgrade-assistant:environments:write` to operate the app, plus `app-engine:apps:install` to install it — both assigned in Account Management
Installation	Install the SaaS Upgrade Assistant app on your target SaaS tenant

> **Tip:** Contact your Dynatrace account team for guided migration support. They can help plan waves and troubleshoot incompatible configurations.

5. What Migrates and What Doesn't

Understanding portability constraints upfront prevents surprises during execution. The SaaS Upgrade Assistant handles most configuration types, but some items require manual recreation.

Portable via SaaS Upgrade Assistant

Configuration Type	Notes
Settings and configurations	Application detection, service detection, data privacy
Management zones	Migrate as-is; consider converting to Grail permissions post-migration
Auto-tagging rules	Tag definitions and conditions
Alerting profiles	Alert filtering and notification routing
Classic dashboards	Migrate automatically; rebuild as modern dashboards recommended
Service detection rules	Custom service naming and merging
Request attributes	Capture rules for request metadata

Configuration Type	Notes
Calculated metrics	Service, log, and RUM calculated metrics
Maintenance windows	Scheduled suppression windows

Requires Manual Recreation

Configuration Type	Why	Recommended Action
Credentials Vault entries	Security — secrets cannot be exported	Re-create in SaaS Credentials Vault
API tokens	Security — tokens are environment-specific	Generate new tokens in SaaS
Webhook endpoints	URLs may differ for SaaS network paths	Reconfigure notification integrations
Synthetic private locations	ActiveGate-bound, environment-specific	Deploy new ActiveGates, recreate locations
Custom extensions (1.0)	EF1 deprecated — must rebuild as Extensions 2.0	Rebuild using Extensions 2.0 framework
Network zones	Environment-specific network configuration	Recreate in SaaS if needed
Plugin-based integrations	Legacy plugin framework	Migrate to Extensions 2.0 or ActiveGate extensions

Non-Portable (Data)

> **Important:** Historic data does **not** migrate. This includes metrics, logs, traces, user sessions, problems, and detected events. Plan for a baseline period where both Managed and SaaS run in parallel so SaaS accumulates its own history.

Data Type	Retention Impact
Metrics	New baseline starts from SaaS go-live
Logs	No historic log data in SaaS

Data Type	Retention Impact
Traces/spans	Distributed traces start fresh
User sessions	RUM session data not transferred
Problems	detected problem history stays in Managed
Deployment events	Release tracking starts fresh

6. Product Safeguards, Limits & SaaS Security Review

During discovery, inventory the product safeguards and platform limits that behave differently on SaaS, and complete a short security and privacy review so nothing surprises you during execution. Several of these settings are **not** carried over automatically by the SaaS Upgrade Assistant and must be re-established manually.

Capture Rates and Traffic Safeguards

Adaptive Traffic Management (ATM) governs how much trace and service-call data is captured. Its behavior depends on your licensing model — confirm which one your target tenant uses before you rely on full trace fidelity.

Licensing model	Capture behavior	Plan for
Classic (host-unit) license	Safeguard caps apply: roughly **250 full-service calls/min per in-use host unit**, an environment floor of **5,000 full-service calls/min**, and per-process bounds of **50–50,000 calls/min**.	Environments near these caps may sample. Compare captured-vs-total call volume before assuming 100% trace fidelity.
Dynatrace Platform Subscription (DPS) / ATM v3	**Adaptive** — sampling adjusts to your licensed volume and typically approaches ~100% capture, with **no fixed per-host-unit number**.	DPS tenants scale capture to licensed volume rather than a static cap. Verify your tenant's ATM version, as capture behavior differs from the classic model.

SQL bind-variable capture is off by default and is a **self-service toggle** — *Settings > Server-side service monitoring > Deep monitoring > Database > "Capture SQL bind values"* (global, with an optional process-group override), **not** a Support request. It is DPS-gated, consumes additional trace ingress volume (which can lower the overall OneAgent capture rate on SaaS), and literal values in batched statements are always masked. Inventory where it is enabled on Managed so you can reproduce it deliberately — and narrowly — on SaaS.

API rate limits — the Dynatrace API returns **HTTP 429** when request-rate limits are exceeded. Bulk automation during migration (token recreation, configuration import, entity queries) should use backoff/retry rather than tight loops.

Settings the SaaS Upgrade Assistant Does Not Carry Over

Some cluster-level safeguards are **not** migrated by the SaaS Upgrade Assistant. Record their Managed values during discovery and re-apply them manually during Prepare/Execute:

Setting	Why it matters	Action
Overload prevention — max entry-point PurePaths/traces per process per minute	Protects against trace floods from a single process	Record the Managed value; set manually on SaaS
Overload prevention — max user actions per minute (RUM)	Caps RUM ingestion spikes	Record and re-apply manually

SaaS Security and Privacy Review

Identity, notification, and data-control surfaces differ on SaaS. Confirm each during discovery:

Area	Managed	SaaS	Implication
SSO federation	SAML, OIDC, LDAP	**SAML 2.0 only** (plus SCIM provisioning)	OIDC federation and direct LDAP are **not** available on SaaS — plan SAML 2.0 through your IdP.
Outbound email (SMTP)	Custom SMTP server (CMC)	Email sent by Dynatrace; **no custom SMTP**	If you need your own mail path, route to an internal relay via a webhook or Workflow email action.

Area	Managed	SaaS	Implication
Data-subject rights	—	**Privacy Rights** app: export personal data and perform **record-level hard deletion in Grail** through an auditable, multi-reviewer workflow	Use this surface for GDPR/CCPA export and deletion requests.
Network access control	—	**IP allow-list** (CIDR) for UI and API	Caveat: it protects the latest (Grail/Gen3) UI and API only — it does **not** block the classic `*.live.dynatrace.com` UI or the data-ingest APIs.
Dynatrace Support access	Customer-grantable	**No customer grant/deny toggle** — role-based, internally approved, restricted to the Dynatrace corporate network with MFA, and **every access and change is audit-logged and visible to you**	Governance shifts from a prospective switch to auditable, least-privilege access.

> **Sources:** Adaptive Traffic Management — classic license (DT docs), Support for SQL bind variables (DT docs), SAML SSO (DT docs), Email workflow action (DT docs), Privacy Rights (DT docs), Record deletion in Grail (DT docs), IP allow-listing (DT docs), Data security controls (DT docs).

7. Step Completion Checklist

Before proceeding to Step 2, confirm that you have completed each item:

Checkpoint	Status
SaaS benefits documented and shared with stakeholders	[ ]
Current environment inventory complete (hosts, services, applications, synthetics)	[ ]
OneAgent version spread assessed (any agents outside support window identified)	[ ]

Checkpoint	Status
ActiveGate inventory documented	[ ]
Migration use cases and goals documented with success metrics	[ ]
SaaS Upgrade Assistant requirements understood (version, IAM, installation)	[ ]
Non-portable items identified and manual recreation plan noted	[ ]
Historic data limitation communicated to stakeholders	[ ]
Product safeguards and limits reviewed (ATM capture behavior, SQL bind-variable capture, API rate limits)	[ ]
Overload-prevention settings inventoried for manual re-apply on SaaS	[ ]
SaaS security review complete (SSO surface, custom SMTP, Privacy Rights, IP allow-list, support-access model)	[ ]

Next Step

> **M2S-02: Step 2 — Strategize** — Define your migration approach, timeline, and risk assessment. Choose between big-bang and phased migration, plan your parallel-run period, and identify dependencies and risks.

Additional Resources

- SaaS Upgrade Assistant Documentation
- SaaS Upgrade Assistant on Dynatrace Hub
- Upgrading from Dynatrace Managed to SaaS
- Grail Data Lakehouse
- DQL Reference

Summary

In Step 1, you:

- Documented the key benefits of Dynatrace SaaS compared to Managed
- Inventoried your current environment (hosts, services, applications, synthetics, ActiveGates)
- Confirmed your migration use cases and goals with measurable success criteria
- Understood the SaaS Upgrade Assistant as the primary migration tool
- Identified what migrates automatically, what requires manual recreation, and what data does not transfer
- Reviewed product safeguards, platform limits, and the SaaS security/privacy surface (capture rates, SSO, notifications, privacy controls, support-access model)
